



MODUL 13 · UPLOAD FILE & OPTIMASI GAMBAR

Validasi File dengan Zod

Storage + CDN + transformasiZod tidak punya `z.file()` bawaan — gunakan `z.instanceof(File).refine()` untuk validasi lengkap. on-the-fly. Free tier 25GB. Production-ready dari hari pertama.



Risiko Upload Tanpa Validasi

Tanpa validasi, aplikasi rentan terhadap berbagai serangan dan penyalahgunaan.



Malware Upload

User upload .exe, .php, .sh — jika dieksekusi bisa kuasai server.



Storage Exhaustion

Upload file 10GB berkali-kali — storage habis, billing meledak.



MIME Spoofing

File .html atau .php di-rename jadi .jpg — bypass cek extension saja.



Path Traversal

Filename '.././../etc/passwd' bisa overwrite file sistem penting.

U P L O A D F I L E & N E X T / I M A G E

Zod File Schema — instanceof + refine()

z.instanceof(File) untuk type check. .refine() untuk size dan MIME type validation.

lib/validations/file.ts — reusable validators

```
import { z } from "zod"

const IMAGE_TYPES = ["image/jpeg", "image/png", "image/webp", "image/gif"] as const
const DOC_TYPES = ["application/pdf",
  "application/vnd.openxmlformats-officedocument.wordprocessingml.document"] as const
const MAX_IMG = 5 * 1024 * 1024 // 5MB
const MAX_DOC = 10 * 1024 * 1024 // 10MB

// — Image validator —
export const imageFileSchema = z
  .instanceof(File, { message: "Harus berupa file" })
  .refine(f => f.size > 0, "File tidak boleh kosong")
  .refine(f => f.size <= MAX_IMG, `Ukuran maksimal 5MB`)
  .refine(f => IMAGE_TYPES.includes(f.type as any),
    `Hanya ${IMAGE_TYPES.join(", ")} yang diizinkan`)

// — Document validator —
export const documentFileSchema = z
  .instanceof(File)
  .refine(f => f.size <= MAX_DOC, "Maksimal 10MB")
  .refine(f => DOC_TYPES.includes(f.type as any), "Hanya PDF dan DOC")

// — Form schema dengan file —
export const uploadAvatarSchema = z.object({
  avatar: imageFileSchema,
  alt: z.string().max(100).optional(),
})
type UploadAvatarInput = z.infer<typeof uploadAvatarSchema>

// — Optional file —
export const optionalImageSchema = z.instanceof(File)
  .optional().nullable()
  .refine(f => !f || f.size <= MAX_IMG, "Maksimal 5MB")
  .refine(f => !f || IMAGE_TYPES.includes(f.type as any), "Format tidak valid")
```

U P L O A D F I L E & N E X T / I M A G E

Integrasi Validasi File di Server Action

safeParse → check success → validated.data sudah type-safe. Return fieldErrors ke UI.

```
'use server'
import { uploadAvatarSchema } from '@lib/validations/file'
import { cloudinary } from '@lib/cloudinary'
import { auth } from '@lib/auth'
import { headers } from 'next/headers'

export async function updateAvatar(formData: FormData) {
  // 1. Auth
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) return { success: false, error: 'Unauthorized' }

  // 2. Extract dari FormData
  const rawData = {
    avatar: formData.get('avatar'), // File | null
    alt: formData.get('alt'), // string | null
  }

  // 3. Validasi dengan Zod
  const validated = uploadAvatarSchema.safeParse(rawData)
  if (!validated.success) {
    return {
      success: false,
      errors: validated.error.flatten().fieldErrors,
      // { avatar: ["Ukuran maksimal 5MB"], ... }
    }
  }

  // 4. validated.data.avatar sudah typed sebagai File!
  const { avatar } = validated.data
  const b64 = Buffer.from(await avatar.arrayBuffer()).toString('base64')
  const result = await cloudinary.uploader.upload(
    `data:${avatar.type};base64,${b64}`,
    { folder: 'avatars', public_id: `user-${session.user.id}`, overwrite: true } )

  await db.user.update({
    where: { id: session.user.id },
    data: { image: result.secure_url },
  })
}
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa `z.string()` TIDAK tepat untuk validasi file upload?

- A) `z.string()` tidak support binary
- B) File adalah Blob object, bukan string. Gunakan `z.instanceof(File)` untuk type safety.
- C) `z.string()` lebih lambat

Q2 — Bagaimana cara validasi file maks 5MB dengan Zod?

- A) `z.file().max(5000000)`
- B) `z.instanceof(File).refine(f => f.size <= 5 * 1024 * 1024, 'Maks 5MB')`
- C) `z.blob().size(5000000)`

Q3 — Apakah cek `file.type` saja sudah cukup untuk memverifikasi tipe file?

- A) Ya — browser set MIME type yang benar
- B) Tidak — MIME bisa di-spoof. Untuk keamanan tinggi, cek magic bytes juga.
- C) Ya — server yang verify MIME

Tugas Mandiri

Buat validation schema lengkap untuk semua jenis upload.

01 `lib/validations/file.ts`

Buat `imageFileSchema`, `documentFileSchema`, `optionalImageSchema`. Gunakan constants untuk `ALLOWED_TYPES` dan `MAX_SIZE`. Export semua.

02 Test Kasus Edge

Test: file kosong → error. File 10MB → error ukuran. File `.html` diupload sebagai image → error tipe. Verifikasi messages yang tepat.

03 Form dengan Errors

Tampilkan validation errors dari Zod di bawah file input. Handle `errors?.avatar?.[0]` dengan optional chaining yang aman.

04 Cover Image

Buat `uploadCoverImage` Server Action: validasi dengan `imageFileSchema` → upload Cloudinary folder 'post-covers' → update `Post.coverImage` di DB.



`refine()` chains berurutan — yang pertama dijalankan dulu. Urutkan dari validasi paling cepat ke paling lambat.

Yang Sudah Kita Pelajari

- ✓ Zod tidak punya `z.file()` — gunakan `z.instanceof(File).refine()` untuk validasi.
- ✓ Validasi minimal: `size > 0`, `size <= MAX_SIZE`, `type` dalam `ALLOWED_TYPES`.
- ✓ `safeParse` → `validated.data` sudah typed sebagai `File` — tidak perlu `type assertion`.
- ✓ MIME type bisa di-spoof — untuk keamanan tinggi, tambahkan cek magic bytes.
- ✓ Pattern sama dengan form validation M11 — konsisten di seluruh project.

→ Selanjutnya: **Bab 4 — next/image & Image Optimization**

